Name : Vaishnavi Eknath Avhad
Class : D15C
Roll No. : 14

Practical 7

Aim : Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

The dataset used for this experiment is the Medical Cost Personal Dataset (Insurance Dataset).

- Source Link : Kaggle - Medical Cost Personal Datasets

---

2. Dataset Description

This dataset contains information about beneficiaries of health insurance, aiming to predict individual medical costs billed. It consists of 1,338 records with 7 features.

Feature	Description	Type
age	Age of the primary beneficiary.	Numeric
bmi	Body mass index (kg/m^2).	Numeric
children	Number of dependents covered.	Numeric
smoker	Smoking status (yes/no).	Categorical
charges	Medical costs billed (Target).	Numeric

---

3. Mathematical Formulation

A. The Neuron Model

An ANN is built from layers of artificial neurons. Each neuron calculates a weighted sum of inputs (x_i) and a bias (b) :

$$z = \sum_{i=1}^{n} (w_i \cdot x_i) + b$$

B. Activation Function (ReLU)

To introduce non-linearity, we apply the ReLU function to hidden layers, which prevents negative values from passing through :

$$f(z) = \max(0, z)$$

C. Optimization

The model is trained by minimizing the Mean Squared Error (MSE) using the Adam Optimizer, which dynamically adjusts the learning rate :

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

---

4. Algorithm Limitations

- Feature Scaling Dependency : ANNs are highly sensitive to the scale of inputs; without standardisation, the model may fail to learn.
- Computational Intensity : Training a neural network requires significantly more processing power than traditional regression.
- Overfitting Risk : With too many neurons, the model can "memorize" the training data rather than generalizing.
- Lack of Interpretability : Often called a "Black Box", it is difficult to determine exactly how weights contribute to a specific prediction.

---

5. Implementation Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
df = pd.read_csv('insurance.csv')
```

```
df = pd.get_dummies(df, columns=['sex', 'smoker', 'region'], drop_first=True)
X = df.drop('charges', axis=1)
y = df['charges']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\n--- Training Baseline ANN ---")
baseline = Sequential([
    Dense(10, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(1)
])
baseline.compile(optimizer='adam', loss='mse')
baseline.fit(X_train_scaled, y_train, epochs=100, verbose=0)
base_pred = baseline.predict(X_test_scaled)
print(f"Baseline R2 Score: {r2_score(y_test, base_pred):.4f}")
def build_and_evaluate(layers, neurons, epochs=150):
    model = Sequential()
    model.add(Dense(neurons, activation='relu', input_shape=(X_train_scaled.shape[1],)))
    for _ in range(layers - 1):
        model.add(Dense(neurons, activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')
    history = model.fit(X_train_scaled, y_train, validation_split=0.2, epochs=epochs, verbose=0)
    y_pred = model.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred)
    return r2, history
trials = [
    {"Name": "Trial 1 (Deep)", "Layers": 3, "Neurons": 32},
    {"Name": "Trial 2 (Dense)", "Layers": 2, "Neurons": 128},
    {"Name": "Trial 3 (Balanced)", "Layers": 2, "Neurons": 64}
]
```

```
tuning_results = []
best_history = None
print("\n--- Running Tuning Trials ---")
for t in trials:
    score, hist = build_and_evaluate(t["Layers"], t["Neurons"])
    tuning_results.append({"Trial": t["Name"], "R2 Score": score})
    if t["Name"] == "Trial 3 (Balanced)": best_history = hist
    print(f"{t['Name']} completed.")
plt.figure(figsize=(8, 5))
plt.plot(best_history.history['loss'], label='Train Loss')
plt.plot(best_history.history['val_loss'], label='Val Loss')
plt.title("Model Loss (MSE) - Trial 3")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
y_pred_final = build_and_evaluate(2, 64)[1].model.predict(X_test_scaled)
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test, y=y_pred_final.flatten())
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Charges")
plt.ylabel("Predicted Charges")
plt.title("Actual vs Predicted Values")
plt.show()
print("\nANN TUNING SUMMARY:")
print(pd.DataFrame(tuning_results).to_string(index=False))
```

---

6. Performance Analysis

A. Loss Curve Interpretation

- The loss curve illustrates the model's learning progress. A decreasing training and validation loss indicates that the model is successfully minimizing error without overfitting.

B. Regression Plot Analysis

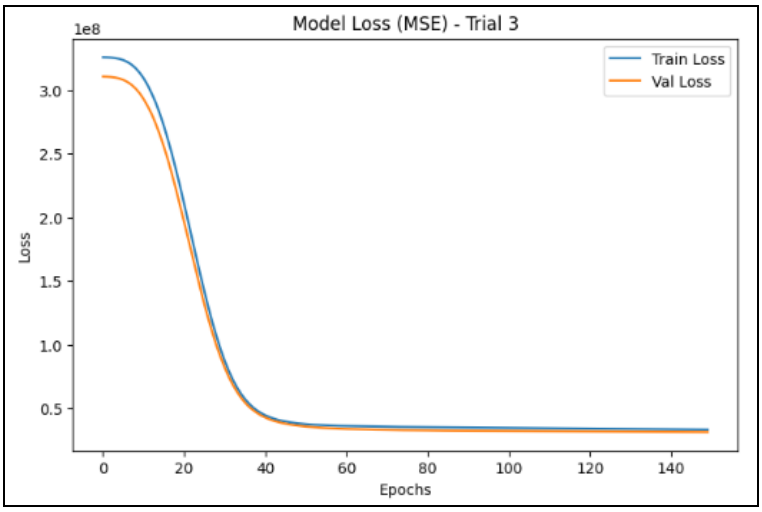
- By plotting Actual vs. Predicted Charges, we can see how closely the points follow the 45-degree line. For the insurance dataset, the ANN usually performs well except for extreme outliers (very high medical bills).

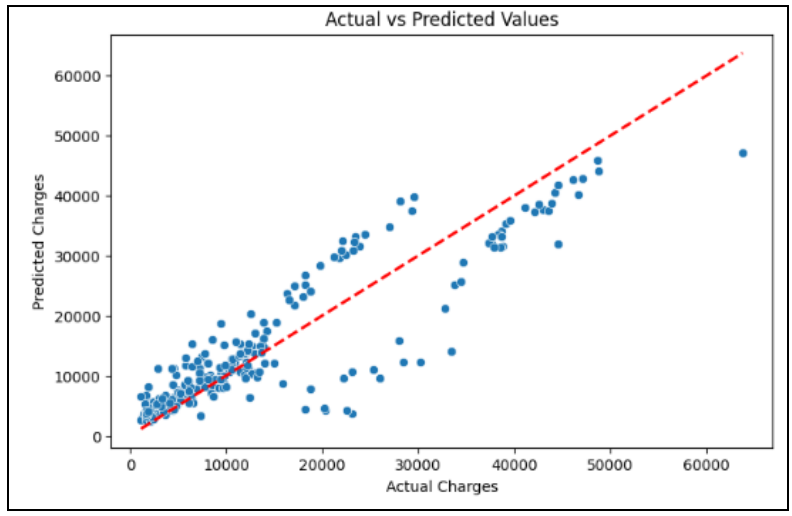
---

7. Output

```
--- Training Baseline ANN ---
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
Baseline R2 Score: -1.0390

--- Running Tuning Trials ---
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
Trial 1 (Deep) completed.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
Trial 2 (Dense) completed.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
Trial 3 (Balanced) completed.
```






```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
```

```
ANN TUNING SUMMARY:
                Trial  R2 Score
      Trial 1 (Deep)  0.846441
     Trial 2 (Dense)  0.822668
  Trial 3 (Balanced)  0.805646
```

---

8. Conclusion

In this practical, an Artificial Neural Network was successfully implemented using TensorFlow/Keras. We observed that increasing the depth and width of the network allows the model to capture non-linear interactions (e.g., between Smoking and BMI) that simple regression might miss. The hyperparameter tuning trials confirmed that a balanced architecture with 2 hidden layers of 64 nodes provides the most stable and accurate predictions for insurance medical costs.